University of Global Village (UGV), Barishal

Department of Computer Science and Engineering

Abstraction in JAVA

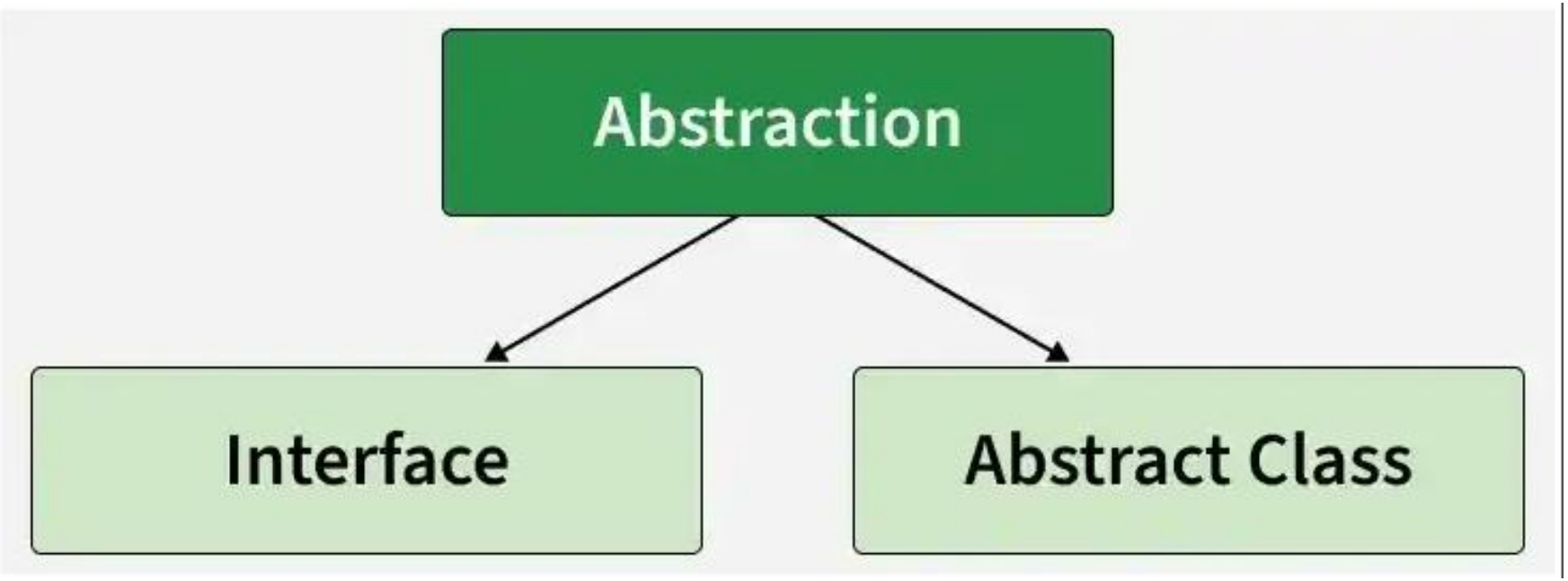
Course Teacher
MD. MEHADI HASAN
Lecturer
Department of CSE
University of Global Village(UGV), Barishal

Abstraction in Java

Abstraction in Java is the process of hiding internal implementation details and showing only essential functionality to the user. It focuses on what an object does rather than how it does it.

- It hides the complex details and shows only essential features.
- Abstract classes may have methods without implementation and must be implemented by subclasses.
- By abstracting functionality, changes in the implementation do not affect the code that depends on the abstraction.

Ways to Achieve Abstraction



Java provides two ways to implement abstraction, which are listed below:

1. Abstract Classes (Partial Abstraction)
2. Interface (provides abstraction for behavior, may contain default or static methods)

Java Abstract Classes

A Java class which contains the **abstract** keyword in its declaration is known as abstract class.

- Java abstract classes may or may not contain *abstract methods*, i.e., methods without body (public void get();)
- But, if a class has at least one abstract method, then the class **must** be declared abstract.
- If a class is declared abstract, it cannot be instantiated.
- To use an abstract class, you have to inherit it from another class, provide implementations to the abstract methods in it.
- If you inherit an abstract class, you have to provide implementations to all the abstract methods in it.

Java Abstract Methods

An **abstract method** is a method that is declared **without a method body**. It only tells the child class **what method should be available**, but it does not define how the method will work.
An abstract method is declared using the abstract keyword.
Syntax:

```
abstract returnType methodName();
```

Why Do We Use Abstract Methods?

Abstract methods are useful when we want different child classes to perform the same operation in different ways.

For example, different ATM booths can have the same operation: withdrawMoney()

But the implementation may be different for different banks.

So the parent class can say: "Every ATM must have a withdrawMoney() method."
The child classes decide how to implement it.

Important Rules of Abstract Methods

1. An abstract method has no body

```
abstract void start();
```

2. An abstract method must be inside an abstract class or interface.

```
abstract class ATM {
    abstract void withdrawMoney();
}
```

3. A child class must implement the abstract method.

```
class DBBLATM extends ATM {

    void withdrawMoney() {
        System.out.println("Money withdrawn from DBBL ATM");
    }
}
```

4. We cannot create an object directly from an abstract class.

```
ATM a = new ATM(); // Error
```

But we can create an object of the child class:

```
DBBLATM a = new DBBLATM();
```

Example 01: ATM Booth System Using Abstract Method

Problem Name: Create an abstract class ATM with an abstract method withdrawMoney(). Create a DBBLATM class that extends ATM and implements the method.

Main.java

```java
abstract class ATM {

    abstract void withdrawMoney();
}

class DBBLATM extends ATM {

    void withdrawMoney() {
        System.out.println("Money withdrawn successfully from DBBL ATM.");
    }
}

public class Main {

    public static void main(String[] args) {

        DBBLATM atm = new DBBLATM();

        atm.withdrawMoney();
    }
}
```

Console | I/O | AI Agent

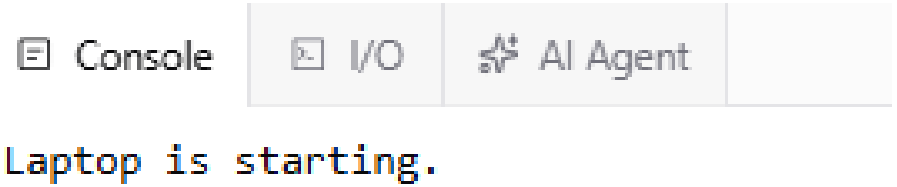
```
Money withdrawn successfully from DBBL ATM.
```

Example 02: Computer Startup System Using Abstract Method

Problem Name: Create a Java program for a Computer Startup System using an abstract method. Create an abstract class Computer with an abstract method start(). Create a subclass Laptop that implements the start() method and displays a message that the laptop is starting.

Main.java

```java
abstract class Computer {

    abstract void start();
}

class Laptop extends Computer {

    void start() {
        System.out.println("Laptop is starting.");
    }
}

public class Main {

    public static void main(String[] args) {

        Laptop laptop = new Laptop();

        laptop.start();
    }
}
```



Problem 03: Create a Java program using an abstract class to represent a general database system. The abstract class should contain an abstract method connect(). Create two subclasses, MySQL and Oracle, and implement the connect() method differently for each database.

Main.java

```java
abstract class Database {

    abstract void connect();
}
class MySQL extends Database {

    void connect() {
        System.out.println("Connecting to MySQL Database");
    }
}
class Oracle extends Database {

    void connect() {
        System.out.println("Connecting to Oracle Database");
    }
}
public class Main {

    public static void main(String[] args) {

        MySQL mysql = new MySQL();
        Oracle oracle = new Oracle();

        mysql.connect();
        oracle.connect();
    }
}
```

What is the role of abstraction here?

The Database class defines what every database should do:

```
abstract void connect();
```

But it does not define how the connection happens. The child classes decide the implementation:

- MySQL → connects to MySQL
- Oracle → connects to Oracle

Output

```
Connecting to MySQL Database
Connecting to Oracle Database

=== Code Execution Successful ===
```

Problem 04: Create an abstract class ATM with an abstract method withdrawMoney(). Create two subclasses, CityBankATM and DBBLATM, and implement the withdrawMoney() method differently in each class.

Main.java

```java
abstract class ATM {

    abstract void withdrawMoney();
}

class CityBankATM extends ATM {

    void withdrawMoney() {
        System.out.println("City Bank ATM: Money withdrawn successfully.");
    }
}

class DBBLATM extends ATM {

    void withdrawMoney() {
        System.out.println("DBBL ATM: Money withdrawn successfully.");
    }
}

public class Main {

    public static void main(String[] args) {

        CityBankATM cityBank = new CityBankATM();
        DBBLATM dbbl = new DBBLATM();

        cityBank.withdrawMoney();
        dbbl.withdrawMoney();
    }
}
```

Why use an abstract class here?

Because all ATMs have some **common operations**,

- Withdraw money
- Deposit money
- Check balance
- Change PIN

But different ATM systems can implement these operations differently.

So the abstract class provides the **common structure**, while the child classes provide the **specific implementation**.

Output:

Console | I/O | AI Agent

```
City Bank ATM: Money withdrawn successfully.
DBBL ATM: Money withdrawn successfully.
```

Problem 5: Create an abstract class ATM with two abstract methods withdrawMoney() and checkBalance(). Create a subclass DBBLATM that implements both methods. Create two objects of DBBLATM and call both methods

Main.java

```java
abstract class ATM {

    abstract void withdrawMoney();
    abstract void checkBalance();
}
class DBBLATM extends ATM {
    void withdrawMoney() {
        System.out.println("Money withdrawn successfully.");
    }
    void checkBalance() {
        System.out.println("Balance checked successfully.");
    }
}
public class Main {

    public static void main(String[] args) {

        DBBLATM atm1 = new DBBLATM();
        DBBLATM atm2 = new DBBLATM();

        atm1.withdrawMoney();
        atm1.checkBalance();

        atm2.withdrawMoney();
        atm2.checkBalance();
    }
}
```

Console | I/O | AI Agent

```
Money withdrawn successfully.
Balance checked successfully.
Money withdrawn successfully.
Balance checked successfully.
```